Liliana Berenguer Jin
25353882

Weather Planner

Internet Applications | Assignment 1 **Word count: 1933 words**

1. Introduction And Requirements

"Weather Planner" is a small reactive Internet Application that takes a destination city and tells the traveller what to pack given its weather forecast. The client runs in the browser using **Vue 3**. It talks to a **Node.js / Express** server, which does the actual work: it calls **OpenWeather's APIs**[1], reduces what comes back into a short summary, and returns that as one JSON object for the client to display.

Before writing any code, I listed the core requirements from the assignment to match each one to the OpenWeather API that could provide the data. In retrospect, this was a really useful starting point for deciding how to structure the **server side** code. Table 1 shows more or less what I concluded from it:

Requirement	OpenWeather API	What it returns
Turn the city name into a location	Geocoding (`/geo/1.0/direct`)	Latitude and longitude of city
Three-day table of temperature, wind speed and rainfall	5-day / 3-hour Forecast (`/data/2.5/forecast`)	~40 readings at 3-hour intervals
Umbrella if rain is forecast + Cold / Mild / Hot packing advice (below 8 °C, 8–24 °C, above 24 °C)	Forecast (same as above)	Rain and temperature per reading
Warning for each pollutant above the "Good" level, with its level and health risk	Air Pollution (`data/2.5/air_pollution`)	AQI (1–5) and the concentration of each pollutant
(Extra feature) Map with temperature + precipitation layers	Weather Maps (`/map/{layer}/{z}/{x}/{y}.png`)	Map tiles for the temperature and precipitation layers
(Extra feature) Get weather for a point clicked on the map	Reverse Geocoding (`geo/1.0/reverse`) Current Weather (`/data/2.5/weather`) + All of the above	The selected location's name, current weather, forecast and air-quality information

Table 1: Requirements and the OpenWeather API used for each

The APIs themselves also set the order of the requests to OpenWeather clear: the **Forecast API** and **Air Pollution APIs** take coordinates rather than a city name, so Geocoding always has to go first.

[1] All of OpenWeather's API documentation: https://openweathermap.org/api

2. System Architecture

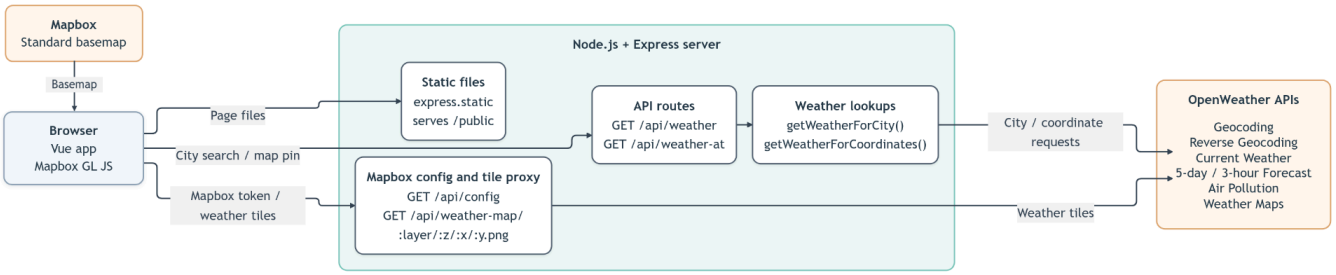


Figure 1: System architecture of the weather app. Arrows show requests between the browser, the Express server, Mapbox and OpenWeather.

Some main observations:

- **Client and server:** Our **server** talks to the browser client in two main ways: (1) it serves the frontend files from `/public`, and (2) it handles the app's API routes. The **Vue client** sends requests to the Express server and displays the returned weather data. The two main weather routes our app has are:
    - **Manual city search:** The browser sends the city name to `GET /api/weather?city=...`. Express calls `getWeatherForCity()`, which: (1) gets the city's coordinates through OpenWeather's **Direct Geocoding API** and (2) passes the location to `getWeatherForCoordinates()`, the shared weather function described below.
    - **Map-pin search:** When the user clicks the map, the browser sends the coordinates to `GET /api/weather-at?lat=...&lon=...`. Express then (1) tries OpenWeather's **Reverse Geocoding API** to get the nearest place's name (to display it onscreen) and (2) passes the original coordinates to `getWeatherForCoordinates()`.
- **Shared weather logic:**
    - getWeatherForCoordinates() is the shared function both paths use. It requests forecast data from OpenWeather's **Forecast API** and current conditions from the **Current Weather API**. We use `Promise.all` because neither request depends on the other: they can run concurrently, reducing the wait before both results are ready.
    - Air-pollution data is requested separately through OpenWeather's **Air Pollution API** . If that request fails, air quality is marked unavailable without failing the whole request.
    - Helper functions such as `makeDailySummaries`, `makePackingAdvice`, `makeAirQualitySummary`, and `makeMapSummary` prepare the data.
- **Map construction flow:** The browser requests `/api/config` to get the public Mapbox token, which **Mapbox GL JS** uses to load the mapbox basemap. For the **temperature** and **precipitation** overlays, Mapbox GL JS requests our `/api/weather-map/:layer/:z/:x/:y.png` route. Express fetches the matching OpenWeather map tile (**Weather Maps API**), using the `temp_new` or `precipitation_new` layer, and returns the PNG tile.
- **State and configuration:** The server doesn't store searches or use sessions; it handles each request independently. The **OpenWeather API key** stays on the server, while the **public Mapbox token** is sent to the browser.

3. Implementation

3.1 Server side

I found myself building the server in three iterations, each one answering a different question: first, can I get anything out of OpenWeather at all; then, what do I do with what comes back; and finally, small refinements after the previous version.

Iteration 1: the first version just called the `Geocoding` and `Forecast` APIs and passed the response straight through, with nothing processed and no error handling. This first version did two things:

- **Chain the two requests**. The handler reads the city from the query string, `awaits` Geocoding for its coordinates, then `awaits` Forecast with them. While it waits, Node's event loop can handle other requests.
- **Return the Forecast response as it comes**, which was useful to see exactly what I was working with. OpenWeather does not return "days" but a list of about forty entries, one every three hours, each with its own temperature, wind and rain values, so I had to group this into 3 days myself.

Iteration 2: This is where the logic of the app actually got built, on top of those responses:

- Daily summaries: The three-hour entries are grouped by **date**, and for each day the server computes **min, max** and **average** temperatures, the average **wind speed**, the total **rainfall** and whether rain is expected.
- Packing advice: The **Cold / Mild / Hot** thresholds form the specification are applied to each day's average, and the advice becomes "pack layers" if the days fall into different categories. The umbrella flag, on the other hand, is set if any day expects rain.
- Air quality: the Air Pollution API call was added. Each pollutant is compared with the upper limit of OpenWeather's "Good" band, and anything above it is reported with its band and a short health risk.
- Error handling: the route checks the city before doing anything, and every failure ends in an HTTP status code[2] and a message the client can show (Table 2).

Status	When it happens
200 (OK)	Success: `city`, `coordinates` ,`currentWeather`, `today`, `days` (up to 3 forecast days), `packing, airQuality, mapSummary` and `hasThreeDays`
400 (Bad Request)	City missing or over 100 characters
404 (Not Found)	Geocoding returns no match
500 (Internal Server Error)	API key missing from `.env`
502 (Bad Gateway)	502 or OpenWeather's own status code

Table 2: Status codes returned by `GET /api/weather`. The client only checks response.ok and shows the message.

[2] For descriptions of HTTP status codes, see Wikipedia's https://en.wikipedia.org/wiki/List_of_HTTP_status_codes

Iteration 3: Once the core requirements were implemented in Iteration 2, I tested the app with more cities and at different times of day, surfacing some of the things that needed fixing:

- Days were grouped on UTC dates, which gave the wrong first day for cities far from Greenwich. Entries are now shifted by `city.timezone` first, and the current partial day is shown separately as "today".
- A failure in the Air Pollution request used to take the whole response down. It now has its own `try/catch`, so the forecast still renders if air quality is unavailable.
- I added the interactive map as the extra feature (see Section 3.4). While adding it, I noticed the `app.get` route was getting long, so I moved the city lookup into `getWeatherForCity()` and the shared weather processing into `getWeatherForCoordinates()`, letting map pins use the same weather logic.
- Finally, I also documented the API using **OpenAPI**. The file `public/openapi.json` now describes the city search (`/api/weather`), map-pin lookup (`/api/weather-at`), map configuration (`/api/config`), and weather-tile route. **Swagger UI** at `/api-docs.html` displays this documentation and lets one try the endpoints.

3.2 Sequence diagram of a search

Figure 2 shows an example of what the final server does when a user searches for a city by typing its name. The browser sends the city to our `/api/weather`. The server then uses fetch with `async/await`: it first calls the **Geocoding** API to turn the city into coordinates, then requests **Current Weather** and **5-day/3-hour Forecast** with the same coordinates in parallel. It also requests **Air Pollution** data in a `try/catch`, so if that call fails, the forecast can still be returned. Finally, after getting the data from OpenWeather, the server groups forecast readings by local day, adds packing and air-quality information, and returns everything as one JSON object. Thus, the browser never sees the OpenWeather responses directly.

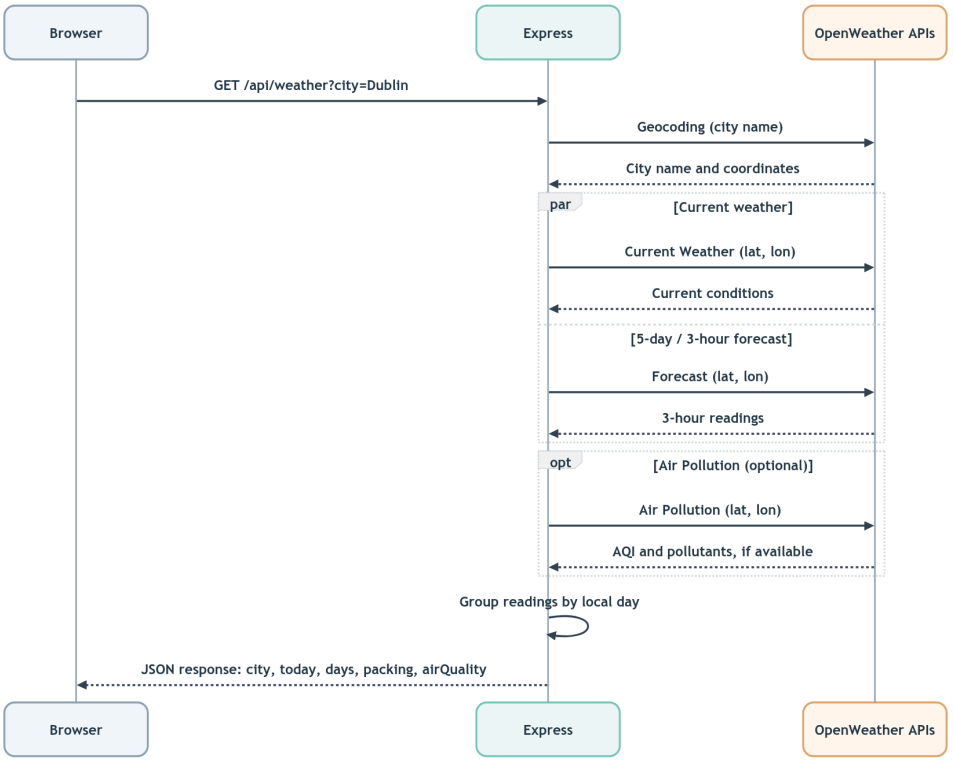


Figure 2: Server-side flow for a single weather search. Express gets the city's coordinates, fetches its forecast and air quality, then returns a daily summary as JSON

3.3 Client side

The client was much more straightforward. Everything lives under `/public`: an `/index.html` with the page structure, `app.js` with the Vue instance, and a separate `style.css`. I started with a bare skeleton (an input, a button and the raw JSON on screen) and built the interface on top of it.

The page holds only a little state: the city typed in, loading status, the result, the map layers, and any errors. The HTML describes what to show for each of those, so when the server replies and the state changes, Vue updates the DOM without me touching it. I used `v-model.trim` for the input, `v-if` to switch between the loading, error and result views, `v-for` for the forecast rows and pollutant warnings, and `:class` for the Cold/Mild/Hot colours. The JavaScript handles city and map-pin requests to the server, the map setup and formatting.

For the layout, today's summary sits on the left, with the forecast table, packing advice and air quality on the right, and an icon for each day's conditions.

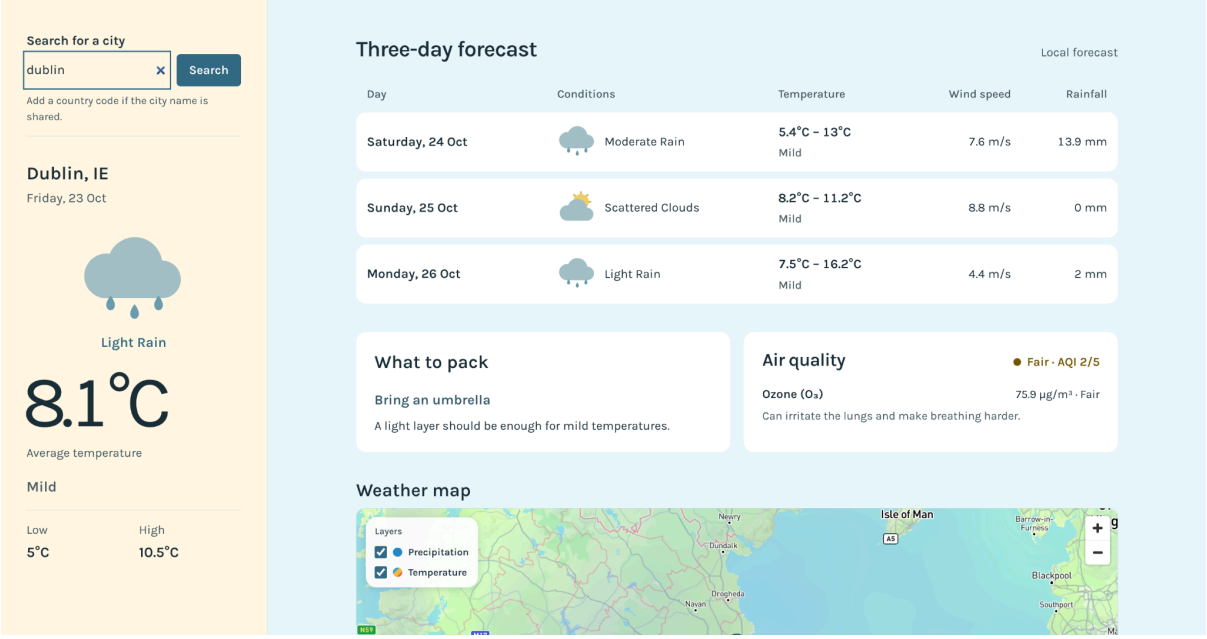


Figure 3: Result View for Dublin, showing current conditions, the three-day forecast, packing advice and air-quality information

3.4 Additional feature: Precipitation + Temperature map

To decide what to add to my app, I thought about the reasons I myself would look at a weather app. I ended up with a simple but, in my opinion, very useful feature: a map that visually shows what we already had in text. I started by looking through OpenWeather's APIs (with the student pack) and found myself in the maps section. OpenWeather provides weather tiles but not a base map, and since I'd already used **Mapbox** before and had a key[3], I went with that. The process looked like this:

[3] Mapbox GL JS uses a public access token to load maps. See its Access Tokens guide: https://docs.mapbox.com/help/dive-deeper/access-tokens/

- First I implemented the **precipitation** layer only. The browser gets the Mapbox token from `/api/config`, Mapbox GL loads the base map, and then it requests the weather tiles from `/api/weather-map/:layer/:z/:x/:y.png` (Figure 1). I didn't want to expose my OpenWeather key, so Express fetches each tile from OpenWeather and passes it on to the browser. Learning how to build this proxy was definitely my biggest lesson here.
- Once I had this working, adding the **temperature** layer was pretty straightforward: same process, as a second layer in `showWeatherMap()`, and I just tweaked the opacities (0.78 for precipitation, 0.4 for temperature) so the base map stays visible.
- A very intuitive next step was dropping a pin on the map. OpenWeather's API already needed coordinates, so I only had to call its **Reverse Geocoding** API to get the place name, as already mentioned, then reuse `getWeatherForCoordinates()` just like the search box does.
- I noticed the **Forecast** API doesn't show current conditions, so I added a **Current Weather** request next to it. It now shows up both when you drop a pin and when you search by text.

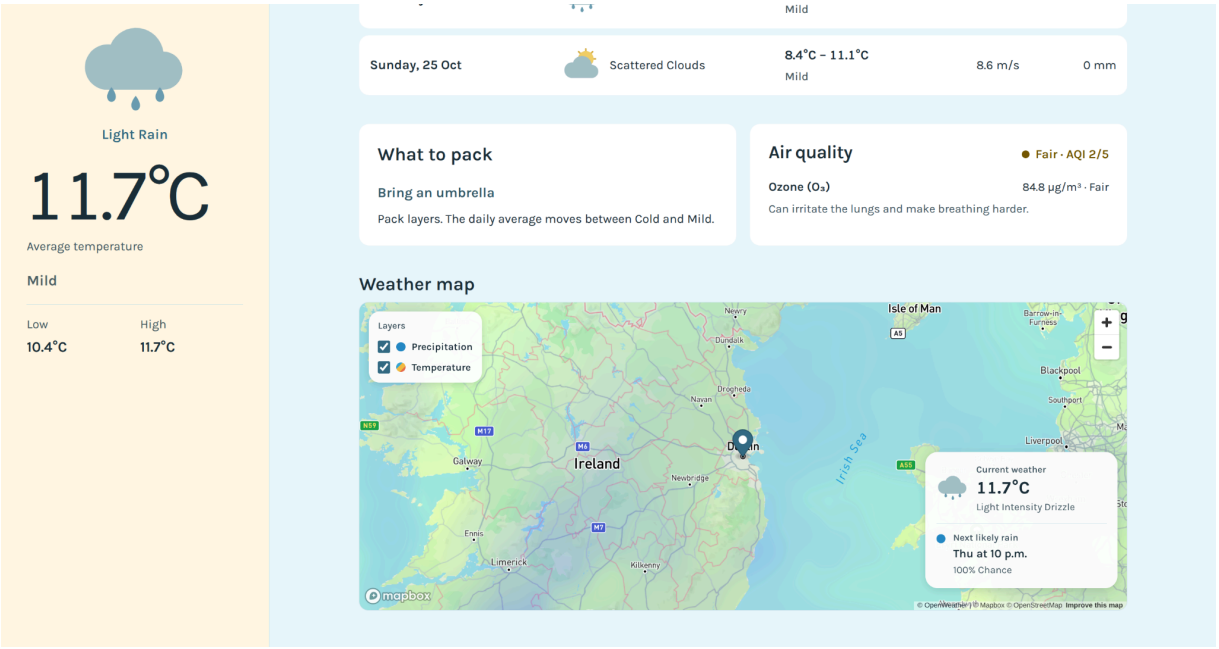


Figure 4: Weather map feature. Shows temperature and/or precipitation overlays, with the forecast and air-quality summary for a selected location.

4. Possible next steps

The obvious next step would be to deploy the server on **AWS**. I could also cache OpenWeather responses for a few minutes, as a single comparison currently makes **four API calls**: Geocoding, Current Weather, Forecast and Air Pollution for each city. Caching could avoid repeating those requests for recent searches. Deploying the app would also mean I could add **automated tests** for the routes and their error responses, a **/health** endpoint so AWS could check that the server is running, and **error logs** to help diagnose problems.

I would also improve the location selection for the coordinates: Geocoding returns several matches for an ambiguous city name, but our app currently uses the first one. A future version could let users choose the correct location, maybe from a drop-down. Additionally, the app currently only shows air-quality for the current reading; a future improvement could be to have it show predicted air quality for the upcoming forecast days, too, making it more useful when planning a trip.